Lecture 35 — Contest Strategy, Debugging, Stress Testing & Team Play

> Year Curriculum · Prerequisites: the whole year so far, especially Week 1 (judges and verdicts), Week 3 (complexity) and Week 18 (problem-solving method) · Estimated time: 2 hours + a practice contest

1. Overview

By now you know a large toolbox of algorithms. This lecture is about converting that knowledge into results when the clock is running. Contest performance depends on more than knowledge:

- **Strategy**: which problems you read, in which order you solve them, when you give up on one.
- **Correct-first implementation**: writing code that is right the first time more often.
- **Debugging**: finding the bug quickly when it is not — through checklists, compiler tools, and, above all, **stress testing**.
- **Team play** (ICPC): three people and one computer.

Everything here is a skill that improves only with deliberate practice. Use every virtual contest in the coming weeks to practice one specific item from this lecture.

2. Scoring formats and what they reward

- **ICPC**: teams are ranked by number of problems solved, ties broken by total **penalty time**: for each solved problem, the minute of acceptance plus 20 minutes for each rejected submission *on that problem*. Unsolved problems cost nothing. Consequences: solve easy problems early (every minute of delay on each solved problem adds to the penalty), and do not submit without testing — a careless WA costs 20 minutes.
- **Codeforces rated rounds**: each problem's value decays with time, and each rejected attempt costs points (Div. 3/4 and educational rounds use ICPC-style penalties instead). Hacks exist, so "passes pretests" is not the end.
- **AtCoder**: points per problem; ties broken by the time of the last accepted submission plus 5 minutes per wrong submission.
- **IOI / olympiads**: partial scoring by subtasks; there is no penalty for submissions, so always submit the solution for the subtasks you can solve.

3. The start of a contest

3.1 Individual contests

Read problem A immediately (it is almost always easy) and solve it. Then skim the remaining statements quickly — titles, constraints, and the first paragraph — to build a mental map. Easy problems are not always in letter order, especially in ICPC sets.

3.2 ICPC

A proven routine for a team of three:

1. Split the set: each person reads about a third (for 12 problems: A–D, E–H, I–L).
2. While reading, write a one-line summary and a difficulty guess on a shared sheet.
3. The person who finds a clearly easy problem first codes it immediately. Aim for the first accepted solution within 10–15 minutes.
4. Watch the scoreboard. A problem solved by many teams in the first half hour is easy, regardless of how it looks to you.
5. Within about 30 minutes, everybody should know the rough shape of every problem, and problems are assigned.

3.3 Triage

Keep a short table of problems: letter, topic guess, constraints, difficulty, estimated coding time, and who owns it. Attempt problems in order of **expected time to accept**, not in letter order and not by personal interest. Re-estimate as you learn more: a "medium" problem that has taken 40 minutes with no idea is now a "hard" one.

4. From statement to algorithm

4.1 Constraints tell you the complexity

The fastest filter for ideas is the size of the input. With ~10^8 simple operations per second:

n	Feasible	Typical techniques
≤ 10–12	O(n!·n)	permutations, backtracking (Week 8)
≤ 20–25	O(2^n·n), O(2^{n/2})	bitmask DP (Week 26), meet in the middle
≤ 100–500	O(n^3)	Floyd-Warshall, interval DP
≤ 5000	O(n^2)	2D DP, all pairs
≤ $2 \cdot 10^5$ – 10^6	O(n log n), O(n√n)	sorting, segment trees, Mo's algorithm

n	Feasible	Typical techniques
$\le 10^7$ – 10^8	O(n)	linear scans, sieve
$\le 10^{18}$	O(log n), O(√n), O(1)	number theory, binary search, matrix power

Also look for hidden constraints: "the sum of n over all test cases does not exceed $2 \cdot 10^5$" means a per-test O(n log n) is fine, but resetting a 10^6-size array per test case is not.

4.2 Before typing

- Re-read the statement, especially the output format and the constraints.
- Run your idea by hand on the samples. If the idea is greedy, try to break it with a small counterexample or find an exchange argument.
- Check extreme cases: n = 1, all values equal, all negative, maximum values, an empty answer.
- Determine value ranges: if a sum can exceed $2 \cdot 10^9$, use `long long`.
- Sketch the code structure on paper. In ICPC this is mandatory: thinking at the keyboard blocks the team's only computer.

4.3 Implementation hygiene

- Start from a short, tested template (Section 8.1).
- Prefer clarity: meaningful names, short functions, no clever macro tricks.
- Default to `long long` for anything that accumulates.
- Size containers from the input. Fixed arrays sized from a guess cause RE (or silent WA when the overflow writes into another array).
- With multiple test cases, reset **all** state at the start of each test, and never `break` out of an input-reading loop early — the unread input will be read as the next test case.
- Avoid copy-paste of nearly identical code (four directions, two symmetric cases): write a loop or a function, otherwise the copies diverge.

5. Debugging

5.1 The WA checklist

Read your code line by line against this list before doing anything else:

1. **Overflow**: `int` products and sums; `1 << k` for k ≥ 31 (use `1LL << k`); intermediate values in modular arithmetic.
2. **Off-by-one**: `<` vs `<=`, 0- vs 1-based indexing, inclusive vs exclusive ranges, loop bounds of binary search.
3. **Multi-test state**: globals, counters and arrays reset; early exit that leaves input unread.

4. **Edge cases**: n = 1, n = 0, duplicates, negative numbers, disconnected graphs, answer "impossible".
5. **Output format**: exact strings ("YES" vs "Yes"), separators, one answer per line, enough decimal digits.
6. **Initialization**: `best = 0` when the answer can be negative; `INF` too small (1e9 when distances reach 10^{14}).
7. **The algorithm itself**: is the greedy proven? does the DP cover all transitions?

5.2 RE, TLE, MLE

- **Runtime error**: out-of-bounds access, stack overflow from deep recursion (a DFS on a path of 10^6 vertices), division or modulo by zero, `.front()`/`.back()`/`.top()` on an empty container, dereferencing `end()`.
- **Time limit**: wrong complexity, but also `endl` (flushes every line), unsynced `cin`, passing large containers by value, `map` where an array works, an infinite loop (which is really a WA).
- **Memory limit**: huge static arrays, memo tables larger than necessary, `vector<vector<…>>` with millions of tiny vectors, recursion depth.

5.3 Compiler tools

Compile locally with:

```
g++ -std=c++17 -O2 -Wall -Wextra -Wshadow -fsanitize=address,undefined -D_GLIBCXX_DEBUG
sol.cpp -o sol
```

- Warnings catch uninitialized and shadowed variables, comparisons between signed and unsigned values, and missing returns.
- **AddressSanitizer** reports out-of-bounds and use-after-free accesses with the exact source line.
- **UndefinedBehaviorSanitizer** reports signed overflow and invalid shifts.
- **`_GLIBCXX_DEBUG`** turns `vector::operator[]` and iterator misuse into immediate errors.

These slow the program down and some judges do not support them; use them locally only.

5.4 Debug output

Use a debug macro that compiles to nothing on the judge (Section 8.1). Print **intermediate** values — the DP table, the BFS order, the state after each step of a simulation — and compare with a hand computation on a sample. Remember to keep debug output on `cerr`, never `cout`.

5.5 Assertions

`assert(condition)` makes the program crash when one of your beliefs is false: an index in range, a binary-search invariant, flow conservation, a sorted input. On the judge this converts a mysterious WA into an RE, which is valuable information (you can even submit with an assertion deliberately to test a hypothesis about the test data, at the cost of a penalty).

6. Stress testing

Stress testing is the single most effective debugging technique in competitive programming. You need three programs:

1. a **generator** that prints a random small test, seeded from the command line so failures are reproducible;
2. a **brute force** that is obviously correct (exhaustive search, direct simulation, $O(n^3)$ instead of $O(n \log n)$) — the simpler, the better;
3. your **solution**.

A loop generates a test, runs both programs, and stops at the first difference, printing the test. Keep tests **small** ($n \le 6$–10, values in a small range): small tests are more likely to hit edge cases like duplicates and negatives, and the failing case is readable by hand.

When the problem has several correct answers, replace the output comparison by a **checker** that reads the input and your output and verifies validity (and, using the brute force, optimality).

Typical results: within seconds the loop finds something like "n = 1, a = [−6]: expected −6, got 0", which points straight at the bug. Section 8.2 contains a complete working example.

Stress testing also works for **TLE**: generate a maximum-size test (and adversarial patterns: sorted, all equal, a path graph, a star graph) and time your solution.

7. Time management and team play

7.1 When to switch

- **The 20-minute rule**: after about 20 minutes without progress on a problem, explain it to a teammate or switch to another problem. Explaining often reveals the gap by itself.
- Re-read the statement: "impossible" problems are often misread ones.
- Change the angle: binary search on the answer, process in reverse, count the complement, model as a graph, look at small cases for a pattern.
- Unsolved problems carry no penalty. Do not let the sunk cost of an hour keep you on a problem that the scoreboard says is hard.

7.2 ICPC roles

With three people and one computer, the computer is the bottleneck. Common roles:

- **Driver**: whoever has a fully planned solution types it. Nobody "thinks" at the keyboard.
- **Thinkers**: solve the next problems on paper and write code outlines, so the next driver can type quickly.
- **Reviewer**: when a submission fails, print the code and debug on paper, letting someone else use the computer meanwhile.

Before every submission a second person checks the sample output and the output format. Agree on a shared problem sheet so two people never accidentally work on the same problem.

7.3 Communication

Use short, factual status updates ("F: idea confirmed, needs about 15 minutes of coding", "H: WA on test 3, printed, debugging"). Decide in advance who sets priorities in the final hour. Keep calm: a rejected submission is information.

7.4 The last hour

The ICPC scoreboard freezes during the last hour. Focus on the problem closest to acceptance. Several people can debug different parts of the same printed code. In the last minutes, submitting a plausible but uncertain solution is free if it is never accepted (penalty only counts for solved problems).

7.5 After the contest

Upsolving — solving, after the contest, the problems you could not — is where most of the learning happens. For each nearly solved problem, read the editorial, close it, and implement the solution yourself. Keep a **mistake log** (what went wrong, how you will detect it next time) and add new, tested code to the team notebook. A good weekly rhythm is one virtual contest, upsolving, and topic practice.

8. Reference implementations

8.1 A contest template

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

#ifdef LOCAL
```

```
#define dbg(x) cerr << "[" << __LINE__ << "] " << #x << " = " << (x) << endl
#else
#define dbg(x)
#endif

void solve() {
    int n;
    cin >> n;
    vector<ll> a(n);
    for (auto& x : a) cin >> x;
    ll sum = 0;
    for (ll x : a) sum += x;
    dbg(sum);
    cout << sum << "\n";
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    cin >> t;  // remove if the problem has a single test
    while (t--) solve();
}
```

Compile locally with `-DLOCAL` to enable `dbg`. Each test case lives in `solve()`, so local variables are automatically fresh; global state must be reset explicitly.

8.2 A complete stress-testing setup

The task: maximum sum of a non-empty contiguous subarray (CSES 1643). The "fast" solution below contains a classic bug.

gen.cpp

```
#include <bits/stdc++.h>
using namespace std;

int main(int argc, char* argv[]) {
    mt19937 rng(argc > 1 ? atoi(argv[1]) : 0);
    auto rnd = [&](int lo, int hi) { return uniform_int_distribution<int>(lo, hi)(rng);
};
    int n = rnd(1, 6);
    cout << n << "\n";
    for (int i = 0; i < n; i++) cout << rnd(-10, 10) << " \n"[i == n - 1];
}
```

brute.cpp — O(n²), obviously correct:

```
#include <bits/stdc++.h>
using namespace std;
```

```cpp
int main() {
    int n;
    cin >> n;
    vector<long long> a(n);
    for (auto& x : a) cin >> x;
    long long best = LLONG_MIN;
    for (int l = 0; l < n; l++) {
        long long s = 0;
        for (int r = l; r < n; r++) {
            s += a[r];
            best = max(best, s);
        }
    }
    cout << best << "\n";
}
```

sol.cpp — Kadane's algorithm with a bug:

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n;
    cin >> n;
    vector<long long> a(n);
    for (auto& x : a) cin >> x;
    long long best = 0, cur = 0;  // BUG: assumes the answer is never negative
    for (long long x : a) {
        cur = max(0LL, cur + x);
        best = max(best, cur);
    }
    cout << best << "\n";
}
```

stress.sh

```sh
#!/bin/bash
g++ -std=c++17 -O2 -o gen gen.cpp && g++ -std=c++17 -O2 -o brute brute.cpp && g++ -
std=c++17 -O2 -o sol sol.cpp || exit 1
for i in $(seq 1 1000); do
    ./gen $i > in.txt
    ./sol < in.txt > out1.txt
    ./brute < in.txt > out2.txt
    if ! cmp -s out1.txt out2.txt; then
        echo "Mismatch on seed $i"
        cat in.txt
        echo "sol:";   cat out1.txt
        echo "brute:"; cat out2.txt
        exit 1
    fi
done
echo "All tests passed"
```

Actual output of running it:

```
Mismatch on seed 7
1
-6
sol:
0
brute:
-6
```

The fix is to require a non-empty subarray: start `best` at the first element (or `LLONG_MIN`) and use `cur = max(x, cur + x)`:

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n;
    cin >> n;
    vector<long long> a(n);
    for (auto& x : a) cin >> x;
    long long best = LLONG_MIN, cur = 0;
    for (int i = 0; i < n; i++) {
        cur = (i == 0) ? a[i] : max(a[i], cur + a[i]);
        best = max(best, cur);
    }
    cout << best << "\n";
}
```

With this version the loop prints "All tests passed".

9. Common mistakes (strategic)

1. **Solving in letter order** instead of by difficulty.
2. **Ignoring the scoreboard** — it is free information about difficulty.
3. **Submitting without running the samples**, or without checking the output format: an avoidable 20-minute penalty.
4. **Staring at code for 30 minutes** instead of writing a brute force and a stress test.
5. **Typing while thinking** in a team contest, blocking the computer.
6. **Not reading all problems**; the easiest problem is sometimes the last one.
7. **Giving up on debugging too early, or too late** — use the 20-minute rule to decide.
8. **Not upsolving**: the contest is only half of the training.

10. Practice problems

Bug-prone classics (practice the checklist and stress testing on them):

- Codeforces 1A — Theatre Square (https://codeforces.com/problemset/problem/1/A) — overflow
- CSES 1068 — Weird Algorithm (https://cses.fi/problemset/task/1068) — overflow in the sequence
- CSES 1083 — Missing Number (https://cses.fi/problemset/task/1083) — sum overflow in `int`
- CSES 1643 — Maximum Subarray Sum (https://cses.fi/problemset/task/1643) — the bug from Section 8.2
- Codeforces 466C — Number of Ways (https://codeforces.com/problemset/problem/466/C) — off-by-one and overflow
- Codeforces 1352C — K-th Not Divisible by n (https://codeforces.com/problemset/problem/1352/C) — math edge cases, verify with brute force
- Codeforces 4C — Registration System (https://codeforces.com/problemset/problem/4/C) — map usage and exact output

Strategy practice (virtual participation, full time limit, keep a triage sheet):

- Codeforces Round 1520 (Div. 3) — https://codeforces.com/contest/1520
- AtCoder Beginner Contest 200 — https://atcoder.jp/contests/abc200
- As a team: any recent ICPC regional problem set from Codeforces Gym or Kattis, 5 hours, one computer

11. Further reading

- Codeforces blog: "How to stress test your solution" and the many "how to debug" posts in the Codeforces catalog
- Antti Laaksonen, *Guide to Competitive Programming* (Springer), chapters 1–2 (introduction, programming techniques)
- Steven Halim, Felix Halim, *Competitive Programming 4*, chapter 1 (tips for being competitive)
- GCC documentation on `-fsanitize` options and libstdc++ debug mode (`_GLIBCXX_DEBUG`)
- ICPC World Finals rules (https://icpc.global/worldfinals/rules) — official scoring and penalty rules